

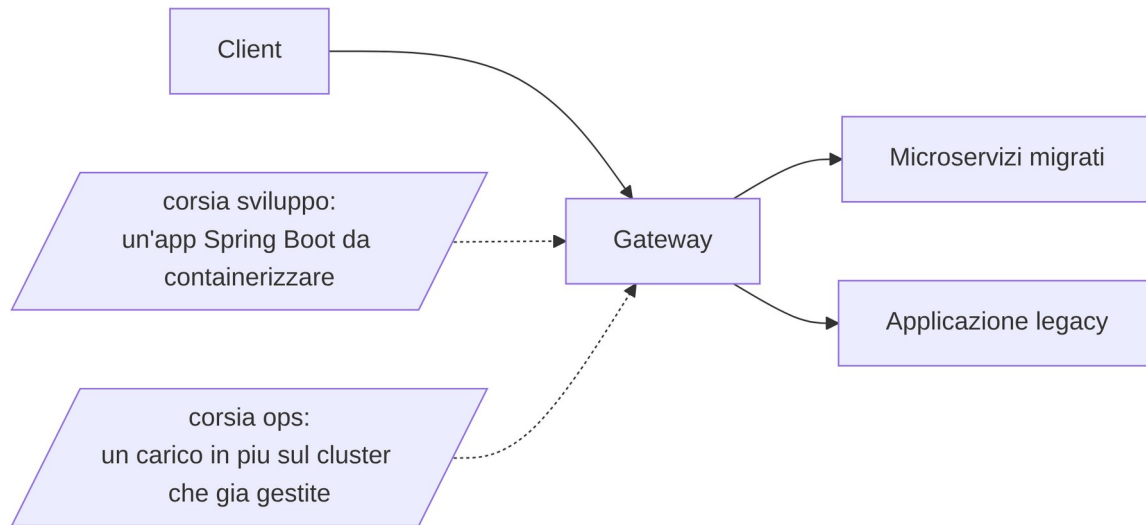
[DRAFT] Sessione 1 — Mettere in piedi il gateway e rendere reale lo split strangler-fig

Obiettivi G1, G2 e primo passaggio su G5

Spring Cloud Gateway per una migrazione strangler-fig su GKE

UNIT 1.1 — DEPLOY DI UN GATEWAY MINIMO SU GKE

Lo stesso gateway è due oggetti diversi secondo il mestiere di chi lo guarda.



Un solo componente, due letture

Finché nessuno raggiunge il gateway dall'esterno, la migrazione non è iniziata.

Il cluster GKE è pronto, l'immagine del gateway esiste. Nessuna richiesta arriva a destinazione.

Il compito è un pod del gateway raggiungibile entro il time box.

- Immagine del gateway già disponibile
- Cluster GKE vuoto e pronto
- Esito atteso: risposta dall'esterno
- Time box dichiarato e visibile

La corsia sviluppo lavora sull'applicazione e sulla sua immagine.

1. Ispezionare l'applicazione Spring Boot
 2. Verificare porta e endpoint di salute
 3. Controllare il Dockerfile
 4. Consegnare l'immagine alla corsia ops
- corsia SVILUPPO*

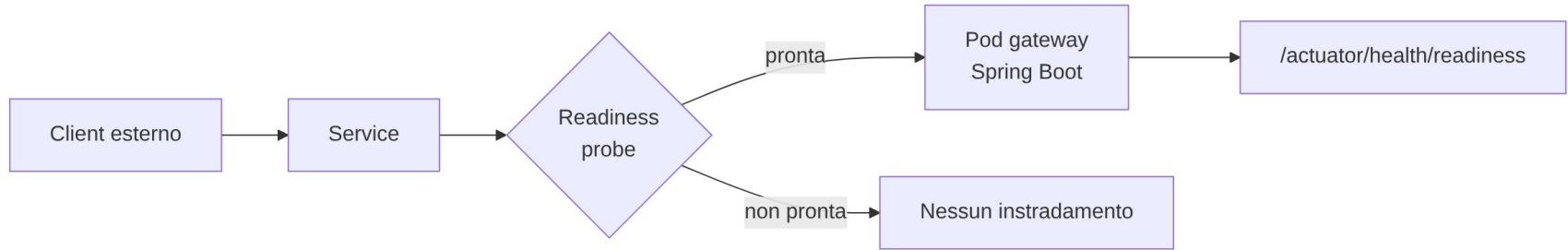
La corsia ops porta l'immagine sul cluster e la rende raggiungibile.

1. Scrivere il Deployment
 2. Esporre il Service
 3. Impostare la readiness probe
 4. Chiamare l'endpoint dall'esterno
- corsia OPS*

Nella maggioranza dei casi il blocco è l'esposizione, non l'immagine.

- Cosa si è rotto per primo?
- Come lo avete capito?
- Chi ha visto cosa, da che punto?
- Cosa cambiereste ora?

Una richiesta raggiunge il pod solo se Service e probe lo consentono.



Il percorso completo di una richiesta

L'exporter OpenTelemetry di default si abilita senza scrivere codice.

```
management:
  tracing:
    sampling:
      probability: 1.0
otel:
  exporter:
    otlp:
      endpoint: http://collector:4317
```

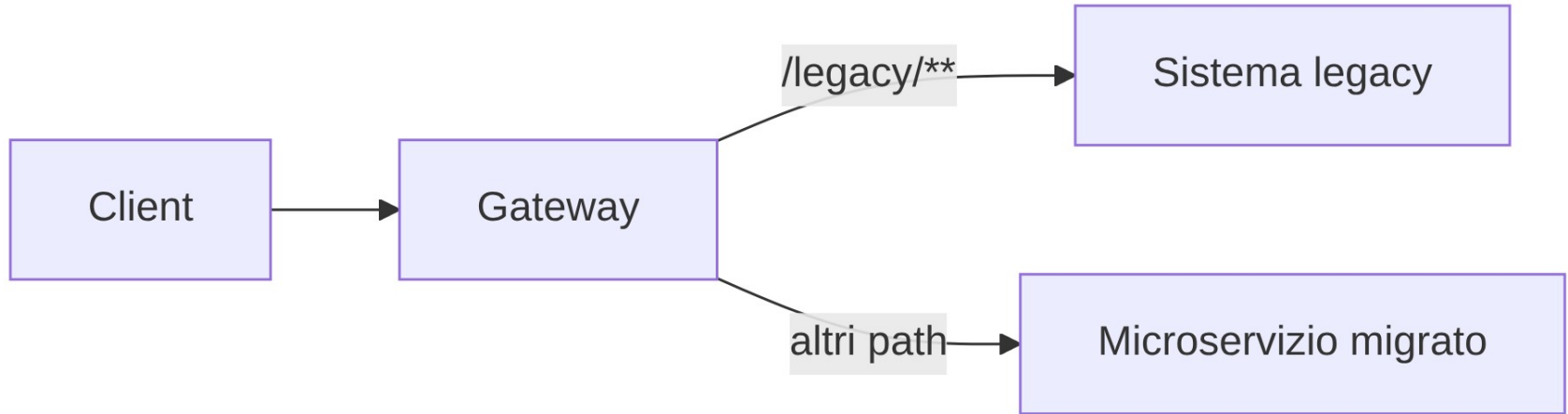
Configurazione, non codice: vale per entrambe le corsie

Una traccia reale della vostra richiesta compare nel collector.

DA FORNIRE: Schermata del collector o della console di osservabilità del cliente con una singola traccia espansa, che mostri lo span del gateway.

UNIT 1.2 — INSTRADAMENTO STRANGLER-FIG BASATO SUL PATH

Ora il traffico deve scegliere fra il servizio nuovo e il legacy.



Lo split che rende reale la strangler fig

Il compito è mandare /legacy allo stub vecchio e il resto al nuovo.

- Due stub già distribuiti e raggiungibili
- Regola attesa: /legacy/** al legacy
- Ogni altro path al microservizio
- Verifica dall'esterno del cluster
- [Documentazione Spring Cloud Gateway — route, predicate, filter](#)

La corsia sviluppo scrive predicate e filter dello split.

```
spring:
  cloud:
    gateway:
      routes:
        - id: legacy
          uri: http://legacy-stub:8080
          predicates:
            - Path=/legacy/**
```

Il punto di partenza da completare

in C:\U\UDDO

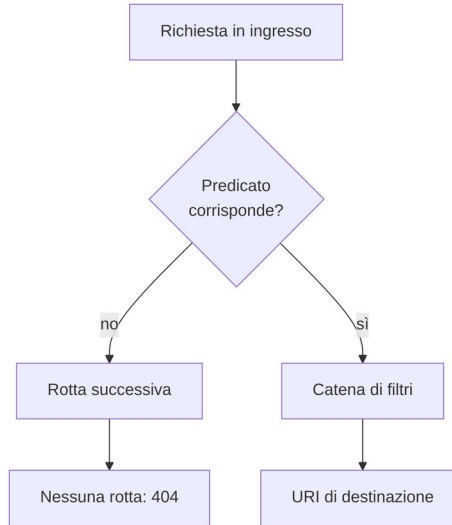
La corsia ops verifica lo split dall'esterno e ne certifica il comportamento.

1. Chiamare un path legacy
 2. Chiamare un path nuovo
 3. Chiamare un path inesistente
 4. Riportare le tre risposte osservate
- corsia OPS*

I casi limite dello split si vedono solo chiamandolo da fuori.

- Priorità fra rotte sovrapposte
- Slash finale presente o assente
- Path che non corrisponde a nulla
- Chi risponde in caso di dubbio

Rotta, predicato e filtro sono i tre soli pezzi dello split.



Come il gateway decide e poi trasforma

Una seconda famiglia di path mette alla prova la regola appena capita.

Estendere l'instradamento a una seconda famiglia di path, decidendo dove collocarla rispetto alle rotte esistenti.

Scegliete un approfondimento fra tre, se il tempo lo consente.

- Instradamento a pesi e canary
- Ingress GKE contro Gateway API
- Filtri di riscrittura del path

Nella settimana che separa le due sessioni ogni corsia prepara la propria.

- Sviluppo: struttura di un token Entra ID
- Sviluppo: endpoint JWKS, a cosa serve
- Ops: collector ancora raggiungibile